

トラヒックエンジニアリングのための深層学習を用いた集中管理型の経路設計法

伊藤 真^{†a)} 見越 大樹^{†b)} 源田 浩一[†]

Path Planning Method using Deep Learning Model for Traffic Engineering

Makoto ITO^{†a)}, Taiju MIKOSHI^{†b)}, and Kouichi GENDA[†]

あらまし ネットワーク内を流れる急激なトラヒックの増減を起因とした輻輳の発生を抑制することは重要な研究課題である。トラヒックエンジニアリングは、このようなトラヒックを制御し、ネットワーク資源を効率的に利用するための技術である。優れたトラヒック制御のためには、複雑な計算を行う必要があり、計算時間の問題を無視することはできない。このような経路設計における諸問題に対して、機械学習技術を適用する研究がある。しかし、従来の機械学習を用いた経路設計法は、拠点間の経路最適化は行える一方で、複数の拠点間を考慮した経路最適化は著しく精度が低下することが分かっている。本論文の目的は、機械学習を用いた輻輳制御のためのネットワーク全体を見通した集中管理型の経路設計法を提案することである。このとき、学習モデルは入力されたトラヒック情報から、全ての拠点間経路全体を出力する必要がある。そこで、本論文では機械学習モデルと教師信号の構成を設計し、実験により性能を評価する。性能評価では、教師信号に Dijkstra 法を用いて学習し、経路設計成功率と帯域使用率、計算時間、トポロジーの変化に対する汎化性能の観点から、Dijkstra 法と比較し、その有効性を示す。

キーワード トラヒックエンジニアリング、輻輳制御、経路設計法、深層学習

1. ま え が き

増加し続けるインターネットトラヒックを起因とする様々な問題や課題への取り組みは重要な研究課題の一つである。そのような問題の一つに、ネットワーク内を流れるトラヒックの時間変動が大きくなることによる、急激なトラヒックの増減を起因とした輻輳の発生がある。一般的なインターネット通信は、ベストエフォート型のサービスであるため、トラヒックの優先度や重要度にかかわらず、輻輳の発生によりトラヒックが破棄されてしまう可能性がある。しかしながら、通信時の経路はトラヒックの送信前に決定されるため、経路を通信途中で制御することはできず、ネットワーク状況に合わせたトラヒック制御が行えないためである。このような問題を解決するための技術として、ト

ラヒックエンジニアリング (TE:Traffic Engineering) がある。TE とは、サービスの品質を維持しながら、時間変動の大きなトラヒックを効率的に制御する方法である [1]。具体的には、サービスの目的や要件に合わせて、多種多様な方式が検討されている [2]~[7]。その中でも本論文では、輻輳回避に焦点を当てる。輻輳回避のための最も簡単なトラヒック制御例は、複数のトラヒックを同一経路ではなく、一部を迂回させ異なる経路を使用することで、ネットワーク資源を効率的に使用することである。そのためには、ネットワーク全体の状況を考慮した経路を設計できる集中管理型の経路設計法を用いる必要がある。また、本論文ではトラヒックの変動に対して、最適な経路を設計するような場面を想定しており、このような場面ではトラヒックデマンドに含まれる全てのフローを同時に再計算する手法が用いられている [8]~[10]。加えて、このような分野では可能な限り短い時間で経路計算を行うことが望ましい。これらは一般的に複雑なアルゴリズムや線形計画法 (LP:Linear Programming) によって求められるため、計算量やスケーラビリティの問題を無視する

[†] 日本大学大学院工学研究科、郡山市
Graduate School of Engineering, Nihon University, 1 Nakagawara, Tokusada,
Tamuramati, Koriyama-shi, 963-8642 Japan

a) E-mail: cemk21002@g.nihon-u.ac.jp

b) E-mail: mikoshi.taiju@nihon-u.ac.jp

DOI:10.14923/transcomj.2023JBP3012

ことができないという課題がある。

このような経路計算における課題の解決に、機械学習技術を用いる研究[11]～[16]が近年見られる。文献[11]～[13]では、目的に合わせて深層学習モデルと教師信号を設計し、教師あり学習により生成した学習モデルを用いて、拠点間トラヒック経路を設計させる方式を提案している。文献[14]～[16]では、周期的にトラヒックの観測を行い、強化学習を用いてルーティングプロトコルに用いられるリンクコストの最適化[14]や、ネットワーク内で高負荷の原因となっているクリティカルフローに対して経路の再計算を行う方式[15],[16]を提案している。機械学習技術を用いる最大の利点は、経路の計算時間を大幅に短縮できることである。複雑なアルゴリズムやLP法を用いた経路設計法では、一般的に膨大な計算時間を要し、最悪の場合は解を得ることができないこともある。そのため、計算時間に関する問題を無視することができない。これに対して、機械学習モデルを用いた場合の計算時間は、ニューラルネットワークの構成や規模に応じて増減するものの、一般的には即時性が高く、非常に短い計算時間である。また、教師信号の作成や学習にかかる時間は長い時間を要することもあるが、あくまでも事前の準備時間であり、経路計算にかかる時間とは直接の関係がない。よって、経路設計アルゴリズムのもつ特徴を正しく学習した機械学習モデルを作成することで、計算時間の課題を解決できる可能性がある。また更なる利点として、サービスの目的や要件に応じた異なる方式やネットワーク機器を導入する必要がないことも挙げられる。学習モデルは教師信号の作成時に用いる経路設計法を適宜変更することで、様々な経路を設計することができる。つまり、機械学習を用いた経路設計法の導入のみで、複数サービスを運用し得ることから、コストの引き下げやサービスの汎用性向上が見込まれる。以上より、機械学習技術を用いることで、従来の経路設計法における計算時間の課題を解決しつつ、様々な要件に対応することができる汎用性の高い経路設計法として期待できる。

しかし、従来の機械学習を用いた経路設計法には、多くの課題がある。一つは、機械学習モデルに設計させる経路を、送信元から宛先ノード間全体とした場合に、学習精度が著しく低下し、十分な経路設計が行えないことである。そのため、文献[11],[12]では、精度向上のために設計する経路情報を次ノードのみとし、ネットワーク内の全ノードが機械学習モデルをもつ

分散型の経路設計としている。また、文献[13]では、送受信間経路全体を出力するために、回帰型ニューラルネットワーク(RNN:Recurrent Neural Network)を用いて、通過すべきノード番号を制約条件として組み込むことで教師信号の種類を減らすことで、学習難易度を低下させ精度の向上を図っている。これらの方式では、ある拠点間トラヒックの経路最適化は図れるものの、複数の拠点間トラヒックを考慮したネットワーク全体の輻輳を回避する経路を設計できない。加えて、文献[14]～[16]では、周期的なトラヒックの観測を前提としており、トラヒックの変化に対して全フローを瞬時に計算できないことや、クリティカルフロー以外の経路の最適化は行えないことから、最悪の場合、輻輳が発生する。よって、本論文の目的である、ネットワーク全体を考慮した複数の拠点間トラヒックからなるデマンド集合に対し、輻輳制御しつつ最短経路を設計するような機械学習を用いた集中管理型の経路設計法は提案されていない。

本論文では、集中管理型の経路設計を行うための深層学習モデルと入出力教師信号の構成を設計する。また教師信号の作成にはTEにおける最も単純な輻輳制御法である、リンクコストを残余帯域の逆数としたDijkstra法を用いることで、Dijkstra法のもつ経路設計に関する特徴量を正しく学習できることを確認する。はじめに、実験用トポロジーを用いて学習を繰り返し、各ハイパーパラメータやデータセット数等を検討し最適なモデルを複数生成する。その後、実験用及び評価用トポロジーを用いて、経路設計成功率、帯域幅使用率、経路計算時間、トポロジーの変化に対する汎化性能の観点から評価を行い、本方式の有効性を示す。

本論文の構成は以下のとおりである。**2.**では、関連研究の概要と課題を述べる。**3.**では、提案方式である機械学習モデル及び教師信号の設計と生成について述べる。**4.**では、提案方式を用いて評価実験を行い、経路設計成功率、帯域幅使用率、計算時間、トポロジーの変化に対する汎化性能の観点から性能評価を行う。**5.**では、本論文の結論と今後の課題について述べる。

2. 関連研究

2.1 Traffic Engineering の課題

トラヒックを起因とする問題の解決を図るための技術が、トラヒックエンジニアリング(TE:Traffic Engineering)である[1]。TEは、ネットワーク資源を効率的に使用できるように、トラヒックが流れる経路を制御

することで、輻輳の回避や帯域幅の節約等の目的を達成とする。実際のネットワークでは、様々な通信方式が混在しているため、対象とするネットワーク内で用いられる各種プロトコルや伝送方式に応じた TE が提案されている [2]～[7]。TE における各経路設計法は、目的や要件に応じて優れた経路を設計できる一方で、計算時間の問題を無視することができない。特に、優れた経路を設計するためには、線形計画法 (LP: Linear Programming) を用いた膨大な計算を行う必要がある。加えて最悪の場合、解を得られず計算が終了しないこともある。更に言えば、トラヒックの時間変動は益々大きくなりつつ、全体のトラヒック量も増え続けていることから、最適経路を設計するための計算難易度は上がり続けている。トラヒックの変動に対して最適な経路を設計するような場面 [8]～[10] では、可能な限り短い時間で経路計算を行うことが望ましく、計算時間に関する課題の解決は重要な要素である。

2.2 深層学習を用いたトラヒック制御法

文献 [11], [12] は、モバイル機器を中心とした異機種間ネットワークにおけるトラヒックの急激な増加を制御する必要性を述べており、機械学習技術を用いたトラヒック制御法を提案している。従来の機械学習では、大規模で異機種間からなるネットワークがもつ動的な特徴を学習することが非常に困難であった。加えて、学習を行う上で最適な学習データをどのように設計すべきか、検討が必要であった。そこで、近年急速に発達している深層学習に焦点を当て、深層学習モデルの各レイヤの構成と学習データの設計を行っている。

この方式では、深層学習を用いたトラヒック制御の実現のために、ネットワーク内の各ルータが深層学習モデルをもち、分散制御する方式を導入している。このとき、各ルータの深層学習モデルへの入力信号は、ルータ自身に流入する直近三つのトラヒックパターンであり、トラヒックパターンは各ルータのインバウンドパケットを用いる。次に、深層学習モデルの出力信号は、経路の設計精度の観点から、送信元ノードから宛先ノード間全体の経路ではなく、次ホップとしてどのルータに送信するかのみ決定させている。なお、集中管理型の送受信間経路全体を設計する方式も検討されたが、最も良い場合でも経路設計成功率が 30% 以下であったことから実現が見送られている。

評価では、Open Shortest Path First (OSPF) による経路と、機械学習による経路を比較している。その結果、機械学習によって設計された経路の方が、シグナリン

グオーバーヘッドが約 70% 削減され、スループットが約 2% 向上し、平均ホップ遅延が約 90% 削減されたと述べられている。その一方で、分散型の経路設計法であることから拠点間における経路最適化は行えるものの、本論文の目的であるネットワーク全体を考慮した経路最適化は行えない。

2.3 回帰型深層学習を用いたトラヒック制御法

文献 [13] は、インターネットトラヒックの爆発的な増加によるネットワークの問題に対処するために、転送制約を考慮した機械学習型トラヒック制御法を提案している。特に、学習対象である経路情報に対して時系列性を見出し、自然言語処理で多く利用されている、Seq2Seq モデルを適用している。Seq2Seq モデルは入力データを有用な情報に変換する Encoder と、変換された情報を再度変換し時系列データを作成する Decoder という、二つのリカレントニューラルネットワークから構成されている。加えて、学習モデルの性能を向上させるために、入力信号と出力信号の各要素の関連性を結びつける Attention メカニズムと、探索アルゴリズムの一種であり、評価値の高い候補補を上位から数個のみ選択することで探索必要な時間や資源を削減する Beam Search を適合し、経路のもつ順次的特徴を捉えることを狙っている。

この方式では、入力信号として送信トラヒックの送信元と宛先ノード番号及び制約条件の三つを用いている。ここで、制約条件とは、必ず通過しなければならないノード番号である。出力信号は送信トラヒックに対する、制約条件を満たしたループのない送受信ノード間の経路全体としている。また、教師信号の作成は二つの工程に分かれており、はじめに Dijkstra 法を用いて、ネットワーク内の全ノード間の静的な経路を探索する。その後、各入力信号に対して事前に計算された静的な経路を組み合わせて、拠点間経路全体を作成し出力信号としている。

評価では、Dijkstra 法による経路と機械学習による経路を比較しており、Dijkstra 法と同様な経路を設計できたと述べられている。また、学習データには含まれていない入力信号に対しても、目的を達成するような経路が設計できることが確認されている。その一方で、ある拠点間トラヒックの最短経路を設計するため、複数トラヒックの経路を設計させた場合に、ネットワーク全体を考慮した経路最適化は行えない。

2.4 TE のための強化学習の適用

文献 [14]～[16] は、トラヒックエンジニアリングに

おける諸問題に対して、強化学習による解決を図っている。文献[14]は、機械学習を用いたTEにおいて、OSPFに用いられるリンクコスト最適化の処理性能向上に貢献している。この方式では、マルチエージェント強化学習(MARL:Multi-Agent Reinforcement Learning)とグラフニューラルネットワーク(GNN:Graph Neural Networks)を組み合わせた学習モデルを用いて、測定されたトラヒックに対して、ネットワーク内の最大負荷を下げるようにリンクコストを再計算させる。文献[15],[16]は、機械学習を用いたTEにおいて、ネットワーク内で高負荷の原因となっているクリティカルフローについて経路の再計算をすることで、ネットワーク内の帯域使用率の改善に貢献している。この方式は、強化学習と線形計画法を用いて、クリティカルフローの再ルーティングを目的としている。文献[14]~[16]の方式は、周期的なトラヒックの観測を前提としており、トラヒックの変化に対して全フローを瞬時に計算することができないことや、クリティカルフロー以外の経路の最適化は行えないことから、最悪の場合、輻輳が発生する。

3. 提案方式

図1は、機械学習を用いた経路設計における一連の流れを表したものである。まず、任意の数の拠点間トラヒックが発生する状況を設定し、これを基にデマンド集合を作成する。次に、デマンド集合に対して任意の経路設計法を用いて教師信号を生成し、学習を行う。学習と評価を繰り返し、十分な性能をもつ学習モデルを生成することで、集中管理型の経路設計が行える。

本論文では、まず教師信号の構成を設計し、これに基づいて深層学習モデルの入力層、出力層、中間層を設計する。次に、学習モデルと教師信号を作成し繰り返し学習を行わせ、一定の性能をもつ学習モデルを複

数生成する。最後に、評価用トポロジを用いて性能評価により学習モデルを選定する。

3.1 教師信号の設計

一般的に、教師信号とは入力データと正解ラベルのことを指し、それぞれの構成を検討する必要がある。なお、本論文では入力データを入力側教師信号、正解ラベルを出力側教師信号と呼称する。

入力側教師信号には、複数の拠点間トラヒックからなるデマンド集合を用いる。このとき、各拠点間トラヒックは、送信元ノード番号、宛先ノード番号、要求帯域幅の三つの情報をもつベクトルで表される。

出力側教師信号には、送受信間経路情報を用いる。このとき、各経路情報は送信元、宛先、中継ノード番号または経路未使用フラグのいずれか一つの情報をもつone-hotベクトルで表される。one-hotベクトルとは、一つの要素のみが1で、他の全ての要素が0であるベクトルであり、教師あり学習における分類問題の出力に用いられるデータ形式である。なお、送受信間経路情報の計算には、目的や要件に合わせて、Dijkstra法や線形計画法などの経路設計法を用いて作成する。

3.2 深層学習モデルの入力層の設計

図2は、入力層の設計についてまとめたものである。入力層の構成を検討する前に、教師信号中に何個の拠点間トラヒックが含まれるかを定める必要がある。仮に、含まれる拠点間トラヒックの最大数が定まらない場合、入力層は無限個の拠点間トラヒックを処理しなければならない、学習しなければならない情報量の際限がなくなり、学習難易度の大幅な上昇を防ぐことができない。これは、計算機資源は有限であるという観点から見ても、適切な設計ではないと言える。そこで、本論文では対象とするトポロジ内全てのノード間で、拠点間トラヒックが一つずつ発生するとする。よって、デマンド集合中に含まれる拠点間トラヒッ

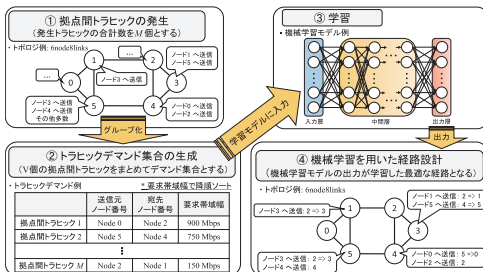


図1 機械学習を用いた経路設計法のシナリオ
Fig. 1 Scenario for path planning using machine learning.

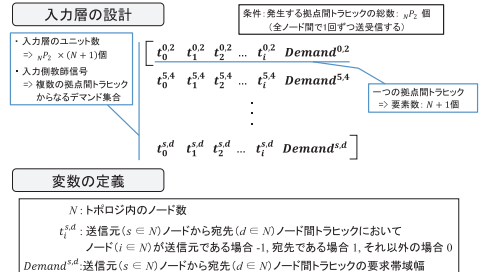


図2 入力層の設計と定義
Fig. 2 Design and define of input layer.

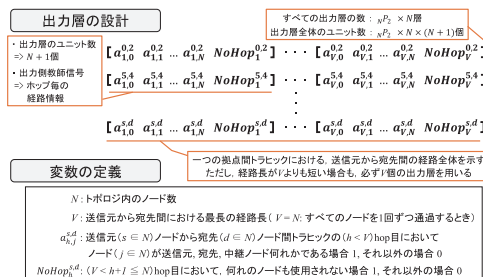


図3 出力層の設計と定義
Fig.3 Design and define of output layer.

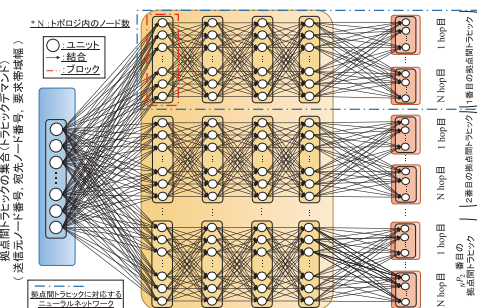


図4 深層学習モデルの構成例
Fig. 4 Deep learning model configuration example.

クの数 は、トポロジ-内 のノ-ド数 を N としたとき、 $N P_2$ 個である。一つ の拠 点間トラヒッ クは $N+1$ 個の要素をもつベクトルで表され、ベクトルの $i (i \in N)$ 番目の要素が -1 であるときノ-ド i は送信元ノ-ドを表し、 1 であるときノ-ド i は宛先ノ-ドを表す。また、それ以外 の 0 である要素は、中継ノ-ドの候補であることを表している。加えて、 $N+1$ 番目の要素は、拠 点間トラヒッ クの要求帯域幅である。以上より、入力層は $N P_2 \times (N+1)$ 個のユニッ トで構成される。

3.3 深層学習モデルの出力層の設計

図3は、出力層の設計についてまとめたものである。提案する深層学習モデルの出力層は、デマンド集合に含まれる各拠点間トラヒックに対して、ホップごとの経路情報を出力する必要がある。ここで、分類問題として学習を行わせるために、ホップごとの経路情報には、送信元ノード番号、中継ノード番号、宛先ノード番号、ノード未使用を表すフラグのいずれか一つのみを示すのが望ましい。そこで、 $N + 1$ 個の要素をもつ **one-hot** ベクトルによって表現する。また、送信元から宛先ノード間において最長である経路長は、トポロジー内の全てのノードを1度ずつ通過するときであり、この数を V とすると、 V はノード数 N と等しい。よって、一つの拠点間トラヒックにおける経路全体を表すためには、 N 個の **one-hot** ベクトルが必要である。ただし、送信元から宛先ノード間の経路が h ホップ ($h < V$) の場合、 $h + 1$ 番目以降は経路情報として使用しないことを示す必要があるため、ノード未使用フラグにより表す。具体的には、**one-hot** ベクトルの j ($j \in N$) 番目の要素が1であるとき、ノード j は送信元ノード、宛先ノード、または中継ノードであることを示し、それ以外の全ての要素を0とする。また、**one-hot** ベクトルの $N + 1$ 番目の要素が1であるとき、この要素はノード未使用フラグを示し、それ以外の全ての要素を

0 とする. 以上より, 出力層は全部で $N\mathbf{P}_2 \times N$ 個であり, 各出力層は $N+1$ 個のユニットで構成される.

3.4 深層学習モデルの中間層の設計

図4は、深層学習モデルの構成例を示している。提案する深層学習モデルの中間層は、複数のブロックから構成され、一つのブロックは入力層と同じ数のユニットをもつ。また、中間層は深さが3層以上で、それぞれの深さで複数のブロックをもつ。このとき、一つの深さにつき N_{P2} 個のブロックをもつ。加えて、ブロックは各拠点間トラヒックごとに連結関係をもつ。なお、一般的に深層学習における中間層は深さが3層以上であり、図4は中間層が4層の場合を示している。

入力層から中間層への結合は、深さが1である全ての中間層のユニットに対して全結合させる。また、ブロックは同一拠点間トラヒックに関する前段と後段のユニットに対して全結合させる。このとき、異なる拠点間トラヒックに関する中間層ブロック同士は一切の結合関係をもたない。そして、中間層から出力層は、各拠点間トラヒックと対応しているブロックの最深から、該当するトラヒックの出力層とのみ全結合させる。ただし、出力層は一つの拠点間トラヒックに対して N 個の **one-hot** ベクトルから構成される。

従来の機械学習を用いた経路設計法との大きな違いは、単一の中間層に全ての経路情報を学習させ、経路設計を行わせるのではなく、中間層と出力層のブロックが、一つの拠点間トラヒックに対する経路情報を学習し、経路設計を行うことである。これにより、単一の中間層を用いて多数の出力側教師信号を学習させた場合に、学習が収束せずに十分な性能を得られないという課題を解決することができる。この課題は、一度に膨大な出力を計算させてしまうことで、出力パターンが非常に複雑化することで、特徴量を十分に学習す

ることができないことが最大の原因である。特に経路設計の場合は、類似している入力側教師信号に対して、全く異なる出力側教師信号が複数存在することもあり、学習が著しく阻害される要因となっている。提案方式では、中間層と出力層をブロック化し学習させる範囲を一つの拠点間トラヒックにおける経路情報に限定することで、学習精度の向上を果たしている。

その一方で、中間層と出力層だけではなく、入力層も一つの拠点間トラヒックのみを入力とする方法も挙げられる。この場合、学習モデルの規模が縮小することで、性能が限られたデバイスでも機械学習を用いた経路設計が行えると考えられる。しかし、この方法では出力パターン数が減少し学習が収束しやすくなる一方で、全く同じ要素からなる入力側教師信号に対して、複数の出力側教師信号を学習することができないという問題がある。つまり、本論文が目的とするネットワーク状況に応じた動的な経路設計が行えない。また、学習に必要な情報量を増やす目的で、入力側教師信号にトラヒックの受け付け順やネットワークの残余帯域などの情報を組み込むこと方法も挙げられる。しかし、この方法では入力側教師信号の種類が、出力側教師信号の種類よりも大幅に増えてしまうことで、入力パターンの複雑化が増し、特徴量を十分に学習することができないという問題がある。以上より、入力側教師信号をトラヒック単位とした場合は、いずれの手段においても学習自体が成り立たず経路設計を行えるような性能をもつ学習モデルを生成することができない。提案方式の場合は、入力側教師信号をデマンド集合とすることで、デマンド集合に含まれる他の送信元、宛先ノード番号や要求帯域幅の情報により、残余帯域などの情報をいわずとも、デマンド集合全体を総合的に収容するような経路を学習できる。

3.5 教師信号の作成と学習

図5は入力側教師信号、図6は出力側教師信号の作成例を表したものである。まず、入力側教師信号は、対象とするネットワークにおいて、 $N \times P_2$ 個の拠点間トラヒックに対して要求帯域順に降順ソートしたものをある一つのデマンド集合とし、ベクトルとして作成する。このとき、各拠点間トラヒックの要求帯域幅は、設定した複数の帯域幅より完全ランダムに割り当てたものとする。次に、出力側教師信号は、各デマンド集合に対して経路設計アルゴリズムを用いて最適経路を計算し、one-hot ベクトルとして作成する。ただし、計算された経路を用いて、全ての拠点間トラヒックを送

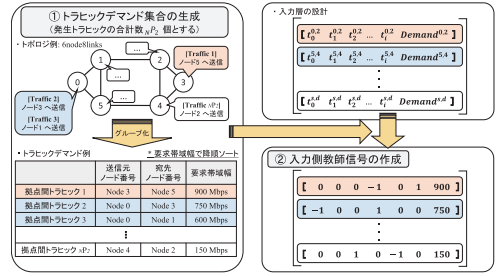


図5 入力側教師信号の生成例

Fig. 5 Generate example of input side teacher signal.

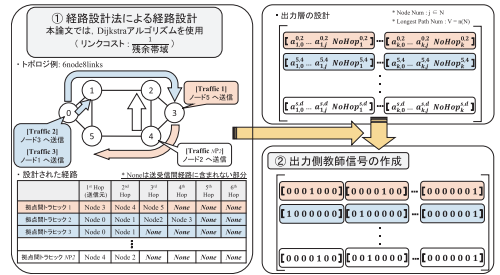


図6 出力側教師信号の生成例

Fig. 6 Generate example of output side teacher signal.

信したとき、リンクのどこか一か所でも輻輳が発生している場合は教師信号として用いない。なお、本論文における教師信号の作成に用いる経路計算アルゴリズムは、リンクコストを残余帯域の逆数とした Dijkstra 法である。これは、IP ベースのトラヒックエンジニアリングにおいてリンクコストを計算するための最も基本的なアルゴリズムであることを理由に採用した。

最後に学習過程について説明する。機械学習を用いる際の重要な要素として、様々なハイパーパラメータの設定や教師信号のデータ数などが挙げられる。本論文では、実験を通して最適化するパラメータと、あらかじめ見当をつけて設定するパラメータの2種類に分けて考える。まず、実験による最適化を行うパラメータは、学習モデルの中間層の層数、教師信号及びテストデータのデータセット数である。本論文において、一つの入力側データセットは $N \times P_2$ 個の拠点間トラヒック情報で構成され、一つの出力側データセットは Dijkstra 法によって計算された当該トラヒックの経路情報で構成される。また、あらかじめ見当をつけて設定するパラメータは、最大学習回数を表す Epoch 数、1回の学習で何個のデータセットを用いるかを表すバッチサイズ、中間層の活性化関数として Relu 関数、出力層の活性化関数として SoftMax 関数、学習精度の指標となる

損失関数は多クラス交差エントロピー、最適化アルゴリズムには Adamax 関数を用いる。なお、学習の繰り返し回数は、十分に大きな値を設定し、一定回数の学習更新がしきい値よりも行われなくなった場合に学習を早期終了する Early Stopping を導入している。

4. 実験評価

評価実験は、以下の六つの実験からなる。実験 1 では、設計に基づいて学習モデルと教師信号を実装し学習を行う。この実験により、中間層の深さの異なる複数の学習モデルを作成し、評価結果から以降の実験で採用する中間層の深さを決定する。実験 2 では、教師信号及び評価用データのデータセット数を複数種類検討し学習させる。この実験により、学習精度と学習時間を考慮した最適なデータセット数を決定する。実験 3 では、これまでに決定した各ハイパーパラメータを用いて複数の学習モデルを実装し学習を行い、実験用トポロジーを用いて提案方式の評価を行う。実験 4 では、実験 3 と同様に複数の学習モデルを実装し学習を行い、評価用トポロジーを用いて提案方式の評価を行う。実験 5 では、実験 4 で用いた複数の学習モデルを用いて、文献 [13] の Seq2Seq 方式と比較評価を行う。実験 6 では、実験 4 で用いた複数の学習モデルを用いて、トポロジーの変化に対する提案方式の汎化性能評価を行う。

4.1 評価条件

評価項目は、経路設計成功率、帯域使用率、計算時間の三つである。はじめに、経路設計成功率は次の式から求められる。

$$\frac{\text{Dijkstra 法による経路と一致している送受信間経路の数}}{NP_2 \times \text{データセット数}} \quad (1a)$$

$$\frac{\text{本論文における成功条件を満たした送受信間経路の数}}{NP_2 \times \text{データセット数}} \quad (1b)$$

評価項目 (1a) は、Dijkstra 法による経路との一致率であり、一般的な機械学習における正答率に該当する。評価項目 (1b) は、本論文における正答率の計算法である。このとき成功条件を満たす経路とは、学習モデルの出力経路において、使用するリンクがネットワーク上に存在し、送信元から宛先まで到達可能であり、ループフリーであり、かつどのリンクにも輻輳が発生していない経路である。次に、帯域使用率は最大負荷

リンクにおける使用帯域幅の各値から求める。本論文では、教師信号作成のための経路設計法として、リンクコストを残余帯域の逆数とした Dijkstra 法を用いており、輻輳制御を目的としている。そこで、学習モデルが教師信号の特性を正しく学習できているかを、帯域使用率の観点から Dijkstra 法と比較評価を行う。最後に計算時間は、提案方式と Dijkstra 法それぞれを用いて、一つのトラヒックデマンド集合の経路を全て計算するまでにかかる時間を 10 回計測し、その平均値により比較評価を行う。ただし、計測時間はデータの事前処理や後処理を含まず、経路計算のみにかかる時間を対象とする。

なお、一般的に機械学習の分野では、教師信号として作成したデータセットを学習用のデータと評価用のデータに分割し、学習に用いたデータでは評価を行わない。これは、ウェブ上やライブラリ中の事前に用意されたデータセットや、実際に運用された履歴情報などを用いることから、データ数に限りがあるためである。その一方で、本論文の想定環境では、デマンド集合に含まれる拠点間トラヒックの組み合わせ数は、全てのノード間の組み合わせ数と要求帯域幅の値、更にはそれらを要求帯域幅の値で降順にソートするため、非常に膨大なパターンが存在する。そのため、教師信号と評価用のデータセットを別々に生成しても、データセット中に同一のデマンド集合が生成される確率は非常に低く、評価実験において未知のデータに対する検証を行えると考えている。よって、教師信号の一部を分割し、学習と評価にそれぞれ用いる方法ではなく、教師信号とは別にデータセットを生成し評価することで、学習モデルが汎用性のある学習を行っているか検討する。なお、要求帯域幅は組み合わせ数に大きな影響を与え、種類が多くなるほど学習難易度が上昇してしまう。よって本論文では、難易度の低減を図るために 150 Mbps から 900 Mbps までの 150 Mbps 刻みの 6 種類に、1 Gbps を合わせた 7 種類からランダムで選択する。この値は、KDDI の帯域保障サービスにおける選択可能な帯域幅を参考にした [17]。

実験に用いるトポロジーは以下のとおりである。実験用トポロジーは複数の 6 ノードトポロジー (図 7) である。これは、実験的な意味合いで用いられる小規模ネットワークトポロジー [18]～[20] であり、実験 1、実験 2、実験 3 にて用いる。また、評価用トポロジーは 6 ノードから 11 ノードの複数のトポロジー (図 8) である。これらは、実際に世界各地で運用経験のある小規

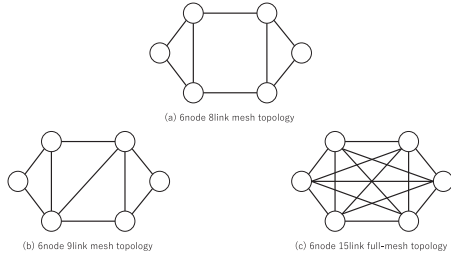


図7 実験用トポロジー
Fig. 7 Topologies only for test.

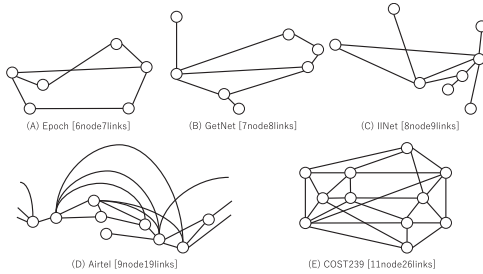


図8 評価用トポロジー
Fig. 8 Topologies only for evaluation.

模なトポロジーであり, Topology Zoo [21] から参照し抜粋し, 実験4, 実験5, 実験6にて用いる. なお, 各トポロジー内の全ての帯域幅は 10 Gbps で統一する. 深層学習モデルの実装と実験は, OS:Ubuntu 20.04.1, 物理メモリ:128 GB, GPU:GeForce RTX 3090, プログラミング言語:Python 3.8.5, Tensorflow2.3.1+Keras を用いて行った.

4.2 実験1 中間層数の検討

表1, 2, 3は, 実験用トポロジーにおける経路設計成功率を示したものである. なお, 経路設計成功率は評価項目(1a)を用いる. この実験で用意した深層学習モデルの構成は, 次のとおりである. 中間層の深さは3層, 4層, 5層. 一度の学習で用いるデータセット数であるバッチサイズは BS:64, BS:128, BS:256. これらを組み合わせた合計9種類とする. また, 教師信号及び評価用データのデータセット数は 10,000 セットとした.

結果より, 全てのトポロジーにおいて中間層の深さが4層及び5層のとき, 高い成功率を示している. この結果から, 実験2以降では深さを4層及び5層を基本として, 適宜ハイパーパラメータの調整を行いつつ高い性能をもつ学習モデルを作成する.

4.3 実験2 データセット数の検討

表4, 5, 6は, 実験用トポロジーにおける平均経路設計成功率を示したものである. なお, 経路設計成功

表1 トポロジー (a) における経路設計成功率 [%]
Table 1 Path planning success rate in topology (a) [%].

中間層	number of batch size		
	BS:64	BS:128	BS:256
中間層 3 層	54.07	32.53	18.80
中間層 4 層	59.13	41.66	21.59
中間層 5 層	54.46	41.24	16.04

表2 トポロジー (b) における経路設計成功率 [%]
Table 2 Path planning success rate in topology (b) [%].

中間層	number of batch size		
	BS:64	BS:128	BS:256
中間層 3 層	74.91	44.60	45.18
中間層 4 層	82.13	77.20	57.55
中間層 5 層	81.33	84.72	52.20

表3 トポロジー (c) における経路設計成功率 [%]
Table 3 Path planning success rate in topology (c) [%].

中間層	number of batch size		
	BS:64	BS:128	BS:256
中間層 3 層	99.76	63.32	65.28
中間層 4 層	99.68	99.85	99.91
中間層 5 層	99.59	99.56	99.69

率は評価項目(1a)を用いる. 表中に記載されている, () 内の数値は全5回の試行における最大値と最小値の差 [%] であり, 成功率の揺らぎである. 実験に用いる教師信号と評価用データのデータセット数は, それぞれ 10,000 セット, 100,000 セット, 1,000,000 セットの3種類とした. また, 中間層は深さを4層とし, バッチサイズを128に固定した.

結果より, 教師信号に着目した場合, 全てのトポロジーにおいてデータセット数が増加するほど, 精度が高くなっていることが分かる. また, 評価用データに着目した場合, 全てのトポロジーにおいてデータセット数が増えるほど, 成功率の揺らぎが小さくなっていることが分かる. ただし, 評価用データの計算時間は, 100,000 セットでは約20分程度に対して, 1,000,000 セットでは約3時間程度を要した. 加えて, 計算時間はトポロジー内のノード数が増加するほど, 急激に増加する傾向が見られるため無視することができない. 以上より, 学習精度, 精度誤差, 計算時間それぞれの観点から総合的に判断し, 実験3及び4以降では教師信号数を1,000,000 セット, テストデータ数を100,000 セットとする.

4.4 実験3 実験用トポロジーを用いた評価

はじめに, 実験3で用いるモデルの学習にかかった Epoch 数を示す. それぞれ最大学習回数 1,000 回に対して, トポロジー (a) は 124 回, トポロジー (b)

表 4 トポロジー (a) における平均経路設計成功率 [%]

Table 4 Average path planning success rate in topology (a) [%].

number of train data	number of test data		
	Test:10,000	Test:100,000	Test:1,000,000
Train:10,000	42.476 (0.01190)	42.697 (0.00205)	42.739 (0.00051)
Train:100,000	85.050 (0.00310)	85.034 (0.00257)	85.061 (0.00011)
Train:1,000,000	97.402 (0.00540)	97.419 (0.00048)	97.421 (0.00036)

表 5 トポロジー (b) における平均経路設計成功率 [%]

Table 5 Average path planning success rate in topology (b) [%].

number of train data	number of test data		
	Test:10,000	Test:100,000	Test:1,000,000
Train:10,000	28.456 (0.08010)	30.073 (0.00351)	30.051 (0.00067)
Train:100,000	90.404 (0.00770)	90.387 (0.00510)	90.425 (0.00025)
Train:1,000,000	97.882 (0.00200)	97.979 (0.00059)	97.986 (0.00002)

表 6 トポロジー (c) における平均経路設計成功率 [%]

Table 6 Average path planning success rate in topology (c) [%].

number of train data	number of test data		
	Test:10,000	Test:100,000	Test:1,000,000
Train:10,000	99.880 (0.00040)	99.908 (0.00087)	99.895 (0.00002)
Train:100,000	99.848 (0.00060)	99.885 (0.00152)	99.862 (0.00003)
Train:1,000,000	99.954 (0.00050)	99.958 (0.00062)	99.954 (0.00005)

は 145 回, トポロジー (c) は 166 回であった。以上より, 全てのモデルにおいて学習は早期終了しており, 学習が収束していることを確認した。図 9 は, 実験用トポロジーにおける拠点間トラヒックごとの経路設計成功率である。トポロジー (a) における評価 (1a) は, 92.23% (2,766,769/3,000,000 トラヒック), 評価 (1b) は, 99.91% (2,997,411/3,000,000 トラヒック) であった。トポロジー (b) における評価 (1a) は, 88.45% (2,653,587/3,000,000 トラヒック), 評価 (1b) は, 99.99% (2,999,970/3,000,000 トラヒック) であった。トポロジー (c) における評価 (1a) は, 99.99% (2,999,961/3,000,000 トラヒック), 評価 (1b) は, 99.99% (2,999,989/3,000,000 トラヒック) であった。また図 10 は, 全ての拠点間トラヒックを流した際の最大負荷リンクにおける使用帯域幅の平均値を比較したグラフであり, 表 7 は, 最大負荷リンクにおける最大値 (Max), 最小値 (Min), 平均値 (Average), 標準偏差 (SD:StandardDeviation), 標準誤差 (SE:StandardError), 中央値 (Median) を記載したものである。ただし, 提案

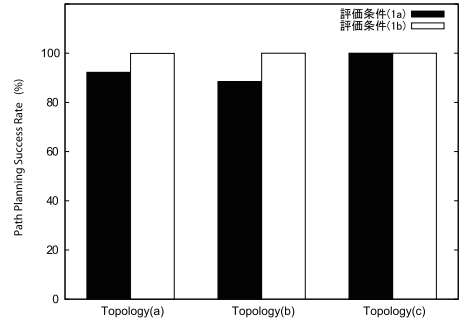


図 9 拠点間トラヒックごとの経路設計成功率

Fig. 9 Path planning success rate by source-destination pairs.

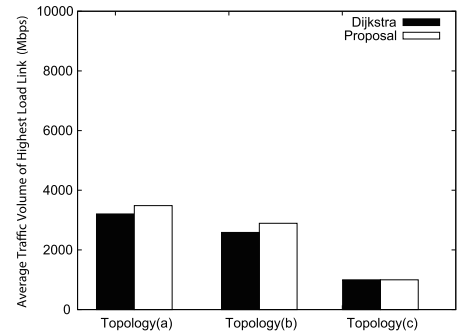


図 10 最大負荷リンクにおける使用帯域幅の平均値

Fig. 10 Average traffic volume of highest load link.

表 7 最大負荷リンクにおける使用帯域幅の詳細値 [Mbps]

Table 7 Detailed traffic volume of highest load link [Mbps].

Topology	Method	Traffic Volume of highest Load Link					
		Max	Min	Average	SD	SE	Median
Topology(a)	Dijkstra	5,600	1,500	3,205.68	429.14	1.357	3,200
	Proposal	5,800	1,650	3,491.68	527.67	1.669	3,450
Topology(b)	Dijkstra	4,300	1,350	2,586.68	341.08	1.079	2,550
	Proposal	4,800	1,500	2,892.12	420.99	1.331	2,850
Topology(c)	Dijkstra	1,000	750	998.99	10.05	0.032	1,000
	Proposal	1,000	750	998.99	10.05	0.032	1,000

方式はデマンド集合に含まれる全てのトラヒックの経路が設計できている場合のみ計算する。これは, 経路設計に失敗している場合, 本来流れるべきトラヒックが流れず計算できないためである。

図 9 より, 教師信号との一致率は全体的に高いものの, トポロジー (a), (b) では約 1 割程度が誤答であった。これは, 少なくとも約 1 割のデマンド集合において, 通信が行えないような経路が含まれていることを表している。これに対して, 本論文における成功条件を満たした経路は, 非常に高い精度で通信可能な経路を設計できている。よって, 学習モデルは教師信号とは異なる経路を設計する場合があるものの, 通信可能

表 8 実験用トポロジーにおける平均計算時間 [s]
Table 8 Path calculation average time in test topology [s].

Method	Topology		
	(a)	(b)	(c)
Dijkstra	0.010	0.010	0.009
Prpposal	0.014	0.014	0.014

な経路を選択しており Dijkstra 法のもつ動的な性質を正しく学習していることが分かった。また図 10 及び表 7 より、帯域使用率の観点から見ると、教師信号とは異なる経路を設計している影響により Dijkstra 法と比較して使用帯域幅の平均値がやや増加しているものの、全てのトポロジーにおいて最大値がリンク容量を超えていないため、輻輳は発生していない。加えて、標準誤差を用いて 99% 信頼区間を求めた。まず、トポロジー (a) のとき、Dijkstra 法では $3,205.68 \pm 3.496$ 、提案方式では $3,491.68 \pm 4.298$ 。トポロジー (b) のとき、Dijkstra 法では $2,586.68 \pm 2.778$ 、提案方式では $2,892.12 \pm 3.429$ 。トポロジー (c) のとき、Dijkstra 法と提案方式共に 998.99 ± 0.082 であった。標準誤差は非常に小さい値であることから、想定環境において性能が大幅に低下するような場面は存在しない。以上より、学習モデルは輻輳制御を目的とした Dijkstra 法がもつリンク選択の動的性質を学習できることが示された。

最後に、表 8 は、 N_{P_2} 個の拠点間トラヒックからなるデマンド集合に対する経路設計時間の平均値である。提案方式は、Dijkstra 法よりもわずかに計算時間を要するものの、ほぼ同等の結果であった。以上より、提案方式は小規模トポロジーに対して経路設計を行う場合に、多項式時間で経路計算が可能な Dijkstra 法とほぼ同等の計算時間であることが示された。

4.5 実験 4 評価用トポロジーを用いた評価

はじめに、実験 4 で用いるモデルの学習にかかった Epoch 数を示す。それぞれ最大学習回数 1,000 回に対して、トポロジー (A) は 322 回、トポロジー (B) は 183 回、トポロジー (C) は 199 回、トポロジー (D) は 285 回、トポロジー (E) は 144 回であった。以上より、実験 3 と同様に全てのモデルにおいて学習は早期終了しており、学習が収束していることを確認した。図 11 は、評価用トポロジーにおける拠点間トラヒックごとの経路設計成功率である。トポロジー (A) における評価 (1a) は、82.54% (2,476,067/3,000,000 トラヒック)、評価 (1b) は、99.97% (2,999,104/3,000,000 トラヒック) であった。トポロジー (B) における評価 (1a) は、95.13% (3,995,642/4,200,000 トラヒック)、

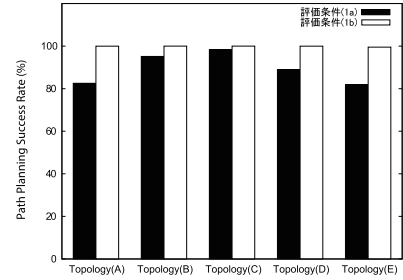


図 11 拠点間トラヒックにおける経路設計成功率
Fig. 11 Path planning success rate by source-destination pairs.

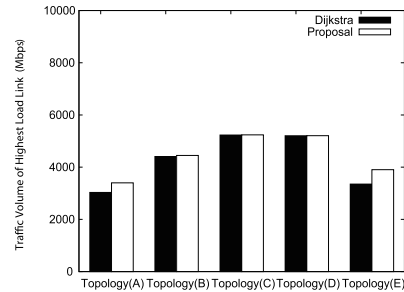


図 12 最大負荷リンクにおける使用帯域幅の平均値
Fig. 12 Average traffic volume of highest load link.

ク)、評価 (1b) は、99.99% (4,199,901/4,200,000 トラヒック) であった。トポロジー (C) における評価 (1a) は、98.26% (5,502,690/5,600,000 トラヒック)、評価 (1b) は、99.99% (5,599,943/5,600,000 トラヒック) であった。トポロジー (D) における評価 (1a) は、88.94% (6,403,509/7,200,000 トラヒック)、評価 (1b) は、99.96% (7,197,219/7,200,000 トラヒック) であった。トポロジー (E) における評価 (1a) は、81.77% (8,995,080/11,000,000 トラヒック)、評価 (1b) は、99.52% (10,946,756/11,000,000 トラヒック) であった。また図 12 は、全ての拠点間トラヒックを流した際の最大負荷リンクにおける使用帯域幅の平均値を比較したグラフであり、表 9 は、最大負荷リンクにおける最大値 (Max)、最小値 (Min)、平均値 (Average)、標準偏差 (SD:StandardDeviation)、標準誤差 (SE:StandardError)、中央値 (Median) を記載したものである。ただし、トラヒックの計算は実験 3 と同様にデマンド集合全体で経路設計に成功した場合のみ計算する。

図 11 より、教師信号との一致率は最大 2 割程度が誤答であり、少なくとも 2 割を超えるデマンド集合において、通信が行えないような経路が含まれていた。これに対して、本論文における成功条件を満たし

表9 最大負荷リンクにおける使用帯域幅の詳細値 [Mbps]
Table 9 Detailed traffic volume of highest load link [Mbps].

Topology	Method	Traffic Volume of highest Load Link				
		Max	Min	Average	SD	SE
Topology(A)	Dijkstra	4,800	1,750	3,032.21	358.32	1.133
	Proposal	5,900	1,950	3,398.79	467.46	1.478
Topology(B)	Dijkstra	6,000	2,700	4,408.95	458.63	1.450
	Proposal	6,450	2,800	4,450.35	454.58	1.436
Topology(C)	Dijkstra	6,900	3,550	5,231.99	453.46	1.434
	Proposal	7,200	3,550	5,236.83	455.51	1.440
Topology(D)	Dijkstra	7,700	2,800	5,204.43	669.93	2.116
	Proposal	7,700	2,800	5,204.48	664.26	2.101
Topology(E)	Dijkstra	5,300	2,250	3,351.98	309.87	0.990
	Proposal	6,950	2,600	3,902.73	460.78	1.457

た経路は、非常に高い精度で通信可能な経路を設計できている。よって、実験用トポロジーと同様に学習モデルは Dijkstra 法の性質を正しく学習できることが分かった。また図 12 及び表 9 より、帯域使用率の観点から見ると、実験 3 と比べて Dijkstra 法との差が全体的に小さくなっているのが分かる。特に、トポロジー (B) や (C) は次数が 1 となるノードが含まれていることや、トポロジー (D) や (E) はノード数、リンク数ともに大きい選択すべき経路が多い。つまり、正しく教師信号の動的性質を学習できていない場合、リンク選択に偏りが生まれ、最大負荷リンクにおける使用帯域幅が増加し、最悪の場合は輻輳が発生する。しかし実際には、トポロジー (B) や (C)、(D) は Dijkstra 法と遜色がない負荷分散が行えており、最も小さな規模であるトポロジー (A) の方がやや劣る結果であった。これは、リンク数の少ないトポロジー (A) は選択可能な経路パターンが少なく、一部のリンクが偏って選択されたためと考えられる。加えて、実験 3 と同様に全てのトポロジーにおいて使用帯域幅の最大値がリンク容量を超えていないため、輻輳が発生していないことが確認された。また、標準誤差を用いて 99% 信頼区間を求めた。まず、トポロジー (A) のとき、Dijkstra 法では $3,032.21 \pm 2.919$ 、提案方式では $3,398.79 \pm 3.808$ 。トポロジー (B) のとき、Dijkstra 法では $4,408.95 \pm 3.736$ 、提案方式では $4,450.35 \pm 3.703$ 。トポロジー (C) のとき、Dijkstra 法では $5,231.99 \pm 3.694$ 、提案方式では $5,236.83 \pm 3.710$ 。トポロジー (D) のとき、Dijkstra 法では $5,204.43 \pm 5.457$ 、提案方式では $5,204.48 \pm 5.411$ 。トポロジー (E) のとき、Dijkstra 法では $3,351.98 \pm 2.524$ 、提案方式では $3,902.73 \pm 3.753$ であった。実験 4 においても、標準誤差は非常に小さい値であることから、評価用トポロジーにおいても想定

表10 評価用トポロジーにおける平均計算時間 [s]
Table 10 Path calculation average time in evaluation topology [s].

Method	Topology				
	(A)	(B)	(C)	(D)	(E)
Dijkstra	0.010	0.021	0.043	0.072	0.199
Proposal	0.014	0.021	0.029	0.053	0.079

環境において性能が大幅に低下するような場面は存在しない。以上より、実験 3 と同様に学習モデルは、現実的なネットワークトポロジーにおいても、輻輳制御を目的とした Dijkstra 法がもつリンク選択の動的性質を学習できることが示された。

最後に、表 10 は、 N_{P2} 個の拠点間トラヒックからなるデマンド集合に対する経路設計時間の平均値である。提案方式は、実験用トポロジーと同じノード数であるトポロジー (A) の場合は、実験 3 と同様にわずかに劣るものの、トポロジー (C) つまりノード数が 8 を超えたところから提案方式の方が優れており、ノード数の増加とともに差が広がっていくのが分かる。以上より、提案方式はトポロジーの規模が大きくなるにつれて、多項式時間で経路計算が可能な Dijkstra 法よりも高速に経路設計できることが示された。

4.6 実験 5 関連研究との比較評価

提案方式の有効性を示すために、2. で述べた関連研究の中から文献 [13] の方式を選出した。文献 [13] は、逐次的に送受信経路全体を設計可能な方式であることから、提案方式の比較方式として適切だと考えた。ここで、文献 [13] では、Seq2Seq モデルに Attention と Beam Search を組み合わせた方式を用いており、以下この方式を Seq2Seq 方式と呼ぶ。比較評価条件は、経路設計成功率並びに最大負荷リンクにおける使用帯域幅とする。加えて、経路設計成功率には評価項目 (1b) を用いる。実験には、最も小さなトポロジーであるトポロジー (A) 並びに、1 番目、2 番目に大きなトポロジーであるトポロジー (D) とトポロジー (E) を用いた。なお、Seq2Seq 方式は一つの拠点間トラヒックに対して送受信間経路を設計することから、本提案との条件を合わせるためにトラヒックデマンド内の全ての拠点間トラヒックに対して、繰り返し経路を設計させたときの結果を用いて比較評価を行う。

表 11 は提案方式と Seq2Seq 方式、それぞれの経路設計成功率である。提案方式は全てのトポロジーにおいて、安定して 99% 以上の精度で経路設計が可能である。その一方で、Seq2Seq 方式は、トポロジーサイズの増加とともに経路設計精度が低下している。これは評価項

表 11 拠点間トラフィックごとの経路設計成功率 [%]

Table 11 Path planning success rate by source-destination pairs [%].

Method	Topology		
	Topology(A)	Topology(D)	Topology(E)
Seq2Seq	99.96	98.79	97.22
Proposal	99.97	99.96	99.52

表 12 最大負荷リンクにおける使用帯域幅の詳細値 [Mbps]

Table 12 Detailed traffic volume of highest load link [Mbps].

Topology	Method	Traffic Volume of highest Load Link					
		Max	Min	Average	SD	SE	Median
Topology(A)	Seq2Seq	10,150	2,500	5,144.94	827.88	2,618	5,050
	Proposal	5,900	1,950	3,398.79	467.46	1,478	3,400
Topology(D)	Seq2Seq	12,700	3,600	6,188.67	945.42	2,990	6,050
	Proposal	7,700	2,800	5,204.48	664.26	2,101	5,200
Topology(E)	Seq2Seq	14,250	4,350	7,262.41	1,075.93	3,402	7,150
	Proposal	6,950	2,600	3,902.73	460.78	1,457	3,850

目 (1b) より、設計された経路のうち輻輳が発生している経路を除外しているためである。表 12 は提案方式と Seq2Seq 方式、それぞれの最大負荷リンクにおける最大値 (Max)、最小値 (Min)、平均値 (Average)、標準偏差 (SD:StandardDeviation)、標準誤差 (SE:StandardError)、中央値 (Median) を記載したものである。提案方式は全てのトポロジーにおいて、輻輳が発生していない一方で、Seq2Seq 方式は輻輳が発生してしまっている。これは、Seq2Seq 方式は教師信号に、リンクコストをホップ数とした Dijkstra 法による最短経路、及び、送信元から制約ノード、制約ノードから宛先までの最短経路を結合し、ループフリーであるものを制限付き経路として学習している。特に、制限付き経路はホップ数が増加し、帯域幅を過剰に使用してしまう。その結果、頻出リンクの使用が過剰になり、輻輳の発生や平均負荷の増加が発生している。加えて、標準偏差からデマンド内のフローの組み合わせやランダムに選択される制限付き経路によって、最大負荷に大きなばらつきがある。これに対して、提案方式はリンクコストを残余帯域の逆数とした Dijkstra 法を学習することで、負荷分散された経路を設計できていることから輻輳の発生を抑制し、平均負荷の低減、最大負荷のばらつきの低減を達成できていることが示されている。以上より提案方式は、本論文の目的であるネットワーク内の最大負荷を低減するような、負荷分散された経路を設計できることが示された。

4.7 実験 6 トポロジーの変化に対する汎化性能評価

機械学習を用いた経路設計では、教師信号作成時のトポロジーに依存した経路設計を行う。そのため文

献 [14] では、リンク故障などの要因でトポロジーが変化した場合に、設計される経路がどの程度汎化性能をもつかについて議論されている。実験 6 では、実験 4 で用意した学習済みの学習モデルを用いて、ネットワーク内でリンク故障が発生した場合に、経路設計成功率にどの程度影響を与えるかについて検証する。実験には、最も小さなトポロジーであるトポロジー (A) 並びに、1 番目、2 番目に大きなトポロジーであるトポロジー (D) とトポロジー (E) を用いた。実験におけるリンク故障の設定は、文献 [14] と同様にした。具体的には、ランダムに N 個のリンク故障が発生することを想定し、故障リンクを変更することで、最大 10 種類のリンク故障パターンを生成する。また、リンク故障数 N は最大で 10 個とし、テストデータは実験 4 で用いたものと全く同じデータセットを使用した。なお、評価条件である経路設計成功率には評価項目 (1b) を用いた。実験により、各リンク故障パターンに対して経路設計成功率を計測し、最大値 (Max)、最小値 (Min)、平均値 (Average) を算出する。ただし、リンク数や故障の組み合わせ数が 10 種類に満たない場合は、可能な限り全ての組み合わせを検証する。

図 13, 14, 15 は、トポロジー (A), (D), (E) における最大 10 個のリンク故障が発生した場合の経路設計成功率の変化を示している。まず、図 13 より、トポロジー (A) では経路設計成功率の極端な劣化が起きている。これは、ノード数 6 に対して、リンク数が 7 であるため、リンク故障によって受ける影響が大きくなることが挙げられる。しかしながら、トポロジーの特性を考慮した場合、妥当な結果であると言える。次に、図 14 より、トポロジー (D) ではリンク故障数の増加に対して段階的に経路設計成功率が低下している。トポロジー (D) はリンクの総数が 19 本であり、全体の 3 分の 1 に該当するリンク故障数 $N = 6$ のとき、平均経路設計成功率が約 50% であり、全体の半分に該当する最大故障数 $N = 10$ のとき、平均経路設計成功率が約 35% 程度であった。原因として、トポロジー (D) は各ノードの次数の差が大きいネットワークであることから、リンク故障の発生する箇所によっては、大きく経路設計成功率が低下する要因となり得る。その一方で、全体の半分のリンクが使用不可になった場合でも、平均で 30% 以上の通信が行えることから、負荷分散によってリンクの偏りが少なくなっており、一定の頑健性を有していると言える。最後に、図 15 より、トポロジー (E) では、リンク故障数の増加に対して緩や

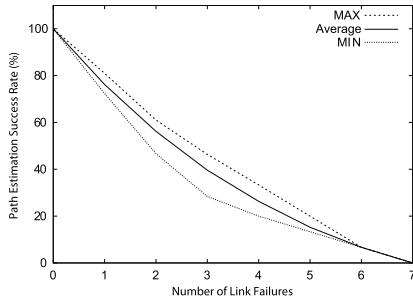


図 13 トポロジー (A) におけるリンク障害に伴う経路設計成功率

Fig. 13 Path estimation success rate by link failures of topology (A).

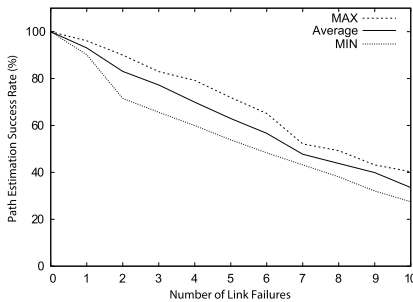


図 14 トポロジー (D) におけるリンク障害に伴う経路設計成功率

Fig. 14 Path estimation success rate by link failures of topology (D).

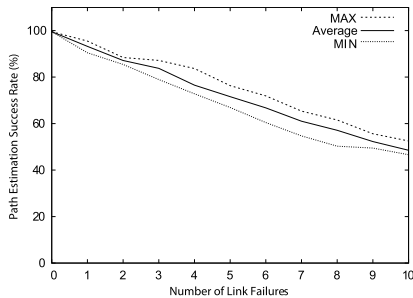


図 15 トポロジー (E) におけるリンク障害に伴う経路設計成功率

Fig. 15 Path estimation success rate by link failures of topology (E).

かに経路設計成功率が低下している。最大リンク故障数 $N = 10$ のとき、平均経路設計成功率が約 50% 程度であり、これまでのトポロジーと比較してリンク故障に対する耐性が高い。要因として、トポロジー (E) は各ノードの次数の差が小さいため、負荷分散された経路の設計により、満遍なくリンクが使用されることで、リンク故障に対する頑健性を得ていると推測できる。

これは、図 15 中の各リンク故障数 N に対して、最大と最小の経路設計成功率の差が、トポロジー (A)、(D) と比較して小さいことから、特定の経路に依存せず、常に一転の頑健性を有していると言える。

以上より、トポロジー (D) や (E) において、平均経路設計成功率が約 50% となるのはリンク故障数が 7 のときであった。また、提案方式による経路は、一つや二つ程度の軽微なリンク故障により、直ちにネットワーク内全ての通信が行えなくなる様な事象は確認できない。よって、提案方式が設計する経路はリンク故障に対して、一定の頑健性を有していると考えられる。

4.8 考察

性能評価より、提案方式の有効性は示された。特に、従来の機械学習を用いた経路設計では、複数の拠点間トラヒックからなるデマンド集合に対して、全ての送受信間経路を一度に計算することは精度の観点から不可能であり、実現されてこなかった。本論文では、教師信号との一致率も高い精度であったのに加えて、ネットワーク上に存在するリンクであり、送受信間経路として使用可能で、ループフリーであり、輻輳がないという条件で経路を評価したところ、非常に高い精度で正しい接続性をもつ経路であることが分かった。また、帯域使用率の観点から中規模程度のトポロジーにおいて、リンクコストを残余帯域分の逆数とした Dijkstra 法と同等レベルの輻輳制御が行えていることが分かった。加えて、従来の機械学習を用いた送受信間経路設計法よりも優れた経路を設計できることを比較評価により示した。次に、経路計算時間の観点では多項式時間で経路設計が可能な Dijkstra 法と比較して、小規模トポロジーの場合は同等程度であり、中規模トポロジーになるにつれて、より短い時間で計算できることが分かった。最後に、これらの経路はリンク故障などのトポロジーの変化に対しても、一定の頑健性を有していると考えられる。この結果から、教師信号を作成する際の経路設計法をより優れた方式を用いることで更なる性能の向上が見込まれる。例えば、線形計画法を用いた経路設計法では、本来非常に長い計算時間を要し、最悪の場合経路が計算できない場合がある。これに対して、提案方式を用いることで線形計画法によって求められる優れた経路を、Dijkstra 法と同等以下の速度で設計できる可能性がある。

また、本実験ではもう一つ優れた結果を得ている。それは、経路設計に成功したデマンド集合において、全ての拠点間トラヒックを流した場合に輻輳するよう

なテストデータは確認されなかったことである。加えて、Dijkstra法を用いて経路計算した際に輻輳が発生するような拠点間トラヒックの組み合わせであるデマンド集合を、学習モデルに入力した場合、経路設計成功率はほぼ0%に近い値で、経路として成り立たないことが分かった。これら二点より、提案方式を用いた経路設計法では、デマンド集合ごとの経路設計成功率の値のみで輻輳制御が可能かどうか判別することができると考えられる。なぜならば、学習モデルの経路設計結果から、高い成功率の場合は輻輳制御が行えるようなデマンド集合であり、低い成功率の場合は輻輳制御が行えないようなデマンド集合であることを判別することができるためである。

今後の課題は大きく三つある。第一に、提案方式が適用可能なトポロジー規模の拡大。第二に、教師信号作成のための最適な経路設計法の検討。第三に、機械学習モデルの構成やハイパーパラメータの規定値の検討。今回の実験では、ノード数11のCOST239では、他のトポロジーに比べてデマンド集合単位でみた場合に経路設計が行えないものが多数確認された。そこで、COST239での更なる精度向上を図りつつ、ノード数12以上のトポロジーにおいても同様の実験を行う。また深層学習モデルの不確実性について検討する必要がある。現在は、深層学習モデルの各種ハイパーパラメータをランダムに変更し、試行回数を増やすことで最適な結果を得ている。その一方で、実験からニューラルネットワークの規模と計算時間には一定の関係性が見られ、加えて各ハイパーパラメータを変化させることで経路設計成功率が増減することが分かった。そこでトポロジーを固定し、各ハイパーパラメータを変化させ、複数の深層学習モデルにより実験を行うことで、経路設計成功率や帯域使用率がどのように変化するか調査を行う。最終的には、ネットワークの規模に応じた、最適な深層学習モデルの構成やハイパーパラメータの規定値について検討する。

5. む す び

本論文では、増加し続ける時間変動の大きなトラヒックの効率的な制御の必要性について説明し、その解決法の一つであるトラヒックエンジニアリングを説明した。一般的にトラヒックエンジニアリングにおける経路設計法は、複雑なアルゴリズムや線形計画法を用いており、最適な経路を計算するためには非常に長い計算時間を要し、最悪の場合経路が計算できないと

いう課題がある。また、トラヒックの変動に対して最適な経路を設計する様な場面では、トラヒックデマンドに含まれる全てのフローを同時に再計算する手法が用いられており、可能な限り短い時間で経路計算を行うことが望ましい。このような経路計算における課題に対して機械学習技術を適用する。そこで、複数の拠点間トラヒックを考慮しつつ、輻輳制御された経路を出力できるような深層学習モデルを検討した。

提案方式では、最も基本的な輻輳制御法であるリンクコストを残余帯域の逆数としたDijkstra法を用いて教師信号を作成し、深層学習モデルに学習させた。評価実験より、小中規模のトポロジーにおいて非常に高い精度で、輻輳制御された経路をDijkstra法と同等以上の速さで設計できることが分かった。また、従来の機械学習を用いた送受信間経路設計法よりも、優れた負荷分散を達成している。特に、提案方式はトポロジーの規模が大きくなるにつれて、Dijkstra法よりも短い時間で経路を計算できることを示した。加えて、リンク故障などを起因とするトポロジーの変化に対しても、一定の頑健性を有していることを示した。以上の結果より、本提案は、トラヒックデマンドの変化があるたびに、瞬時に全てのトラヒックに対してネットワーク全体の負荷分散を考慮した経路設計を行うことができる。

今後の最も重要な課題は、より大規模なトポロジーにおいても輻輳制御された経路を高い精度で設計することである。また、教師信号の作成に用いる経路設計法を線形計画法に変更することで、より優れた経路を学習し設計できるような深層学習モデルを提案する。加えて、適用するトポロジーの規模に応じた、深層学習モデルの構成やハイパーパラメータの規定値について検討し、深層学習モデルの導入難易度低減を図る。

文 献

- [1] N. Wang, K.H. Ho, G. Pavlou, and M. Howarth, "An overview of routing optimization for internet traffic engineering," IEEE Communications Surveys and Tutorials, vol.10, no.1, pp.36–56, First Quarter 2008. doi: 10.1109/COMST.2008.4483669
- [2] M. Pioro, et al., "On open shortest path first related network optimization problems," J. Combinatorial Optimization, vol.48, pp.201–223, April 2002. doi: 10.1016/S0166-5316(02)00036-6
- [3] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," Proc. IEEE INFOCOM 2000. Conference on Computer Communications. Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies (Cat. no.00CH37064), Tel Aviv, Israel, vol.2, pp.519–528, 2000. doi: 10.1109/INFCOM.2000.832225
- [4] D. Mitra and K.G. Ramakrishnan, "A case study of multiservice, multipriority traffic engineering design for data networks," Seam-

- less Interconnection for Universal Services. Global Telecommunications Conference. GLOBECOM'99. (Cat. no.99CH37042), Rio de Janeiro, Brazil, vol.1b, pp.1077–1083, 1999. doi: 10.1109/GLOCOM.1999.830281
- [5] Y. Lee, et al., “A framework for the control of wavelength switched optical networks (WSON) with impairments,” IETF Internet draft, draft-ietf-ccamp-wson-impairments-07, April 2011.
- [6] H. Yonezu, K. Kikuta, D. Ishii, S. Okamoto, E. Oki, and N. Yamanaka, “QoS aware energy optimal network topology design and dynamic link power management,” 36th European Conference and Exhibition on Optical Communication, Turin, Italy, pp.1–3, 2010. doi: 10.1109/ECOC.2010.5621098
- [7] D. Ishii, K. Nakahara, S. Okamoto, and N. Yamanaka, “A novel IP routing/signaling based service provisioning concept for ubiquitous grid networking environment,” 2010 IEEE Globecom Workshops, Miami, FL, USA, pp.1746–1750, 2010. doi: 10.1109/GLOCOMW.2010.5700240
- [8] S. Kandula, D. Katabi, B. Davie, and A. Charny, “Walking the tightrope: responsive yet stable traffic engineering,” Proc. ACM SIGCOMM 2005, pp.253–264, Aug. 2005. doi: 10.1109/GLOCOM.1999.830281
- [9] A. Elwalid, C. Jin, S. Low, and I. Widjaja, “MATE: MPLS adaptive traffic engineering,” Proc. IEEE INFOCOM 2001. Conference on Computer Communications. Twentieth Annual Joint Conference of the IEEE Computer and Communications Society (Cat. no.01CH37213), Anchorage, AK, USA, vol.3, pp.1300–1309, 2001. doi: 10.1109/INFCOM.2001.916625
- [10] T. Otoshi, et al., “Traffic engineering based on stochastic model predictive control for uncertain traffic change,” 2015 IFIP/IEEE International Symposium on Integrated Network Management (IM), Ottawa, ON, Canada, pp.1165–1170, 2015. doi: 10.1109/INM.2015.7140461
- [11] N. Kato, et al., “The deep learning vision for heterogeneous network traffic control: Proposal, challenges, and future perspective,” IEEE Wireless Communications, vol.24, no.3, pp.146–153, June 2017. doi: 10.1109/MWC.2016.1600317WC
- [12] N. Kato, “The deep learning vision for heterogeneous networks control –proposal, challenges, and future perspective–,” The 2017 International Conference on Wireless Internet (WICON' 17), Dec. 2017.
- [13] Y. Zuo, Y. Wua, G. Min, and L. Cui, “Learning based network path planning for traffic engineering,” Future Generation Computer Systems, vol.92, pp.59–67, 2019. doi: 10.1016/j.future.2018.09.043
- [14] G. Bernardes, et al., “Is machine learning ready for traffic engineering optimization?,” 2021 IEEE 29th Int. Conf. Network Protocols (ICNP), Dallas, TX, USA, pp.1–11, 2021. doi: 10.1109/ICNP52444.2021.9651930
- [15] J. Zhang, M. Ye, Z. Guo, C.-Y. Yen, and H.J. Chao, “CFR-RL: Traffic engineering with reinforcement learning in SDN,” IEEE J. Selected Areas in Communications, vol.38, no.10, pp.2249–2259, Oct. 2020. doi: 10.1109/JSAC.2020.3000371
- [16] M. Ye, J. Zhang, Z. Guo, and H.J. Chao, “DATE: Disturbance-aware traffic engineering with reinforcement learning in software-defined networks,” 2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS), Tokyo, Japan, pp.1–10, 2021. doi: 10.1109/IWQOS52092.2021.9521343
- [17] 株式会社 KDDI 国内イーサネット専用サービスメニュー (帯域選択型), <https://www.kddi.com/business/network/intranet/ethernet-senyo/service/menu2/>
- [18] Á. Rubio-Largo, et al., “Multiobjective swarm intelligence for the traf c grooming problem,” Computational Optimization and Applications, vol.60, no.2, pp.479–511, March 2015. doi: 10.1007/s10589-014-9682-8
- [19] I. Popescu, B. Ušćumlić, A. Triki, Y. Pointurier, A. Gravey, and P. Gravey, “Scalable routing, scheduling and virtualization for TWIN optical burst switching networks,” 2015 20th European Conference on Networks and Optical Communications - (NOC), London, UK, pp.1–6, 2015. doi: 10.1109/NOC.2015.7238609
- [20] J. Vales-Alonso, et al., “Performance analysis of optimal schedulers in single channel dense radio frequency identification environments,” EURASIP Journal on Embedded Systems 2013, Jan. 2013. doi: 10.1186/1687-3963-2013-11
- [21] The Internet Topology Zoo, <http://www.topology-zoo.org/index.html>

(2023 年 4 月 14 日受付, 8 月 19 日再受付,
9 月 25 日早期公開)



伊藤 真 (学生会員)

2019 日本大学工学部情報工学科卒。2021 同大学院工学研究科情報工学専攻博士前期課程了。現在、同大学院工学研究科情報工学専攻博士後期課程在学中。機械学習を用いた高信頼ネットワークに関する研究に従事。



見越 大樹 (正員)

2007 日本大学工学部情報工学科卒。2009 同大学院工学研究科情報工学専攻博士前期課程了。2012 同大学院工学研究科情報工学専攻博士後期課程了。博士(工学)。現在、日本大学工学部情報工学科准教授。現在の主たる研究分野、P2P ネットワーク、IoT ネットワークの研究。IEEE、日本バーチャルリアリティ学会、情報処理学会各会員。



源田 浩一 (正員：フェロー)

1990 東北大学大学院工学研究科修士課程了。同年日本電信電話株式会社 (NTT) 入社。2016 日本大学工学部情報工学科教授。現在に至る。SDN やアドホックネットワークなどの情報通信ネットワークの管理・制御技術、IoT デバイスや AI のネットワーク活用技術の研究に従事。IEEE 会員、博士(工学)。